Practical Work 2: RPC File Transfer

Tran Tuan Vu - 23BI14459

December 11, 2025

1 Service Design

We designed the RPC service using the ONC RPC standard. The core of the design is the IDL (Interface Definition Language) file, which defines a structure capable of holding file metadata and content.

The protocol defines a single remote procedure `UPLOAD_FILE` which accepts a `file_data` structure containing the filename, the raw binary data (opaque), and the data length.

```
struct file_data {
    string filename<256>;
    opaque content<>;
    int data_len;
};

program FILE_TRANSFER_PROG {
    version FILE_TRANSFER_VERS {
        int UPLOAD_FILE(file_data) = 1;
    } = 1;
} = 0x31230000;
```

Listing 1: IDL Definition (transfer.x)

2 System Organization

The system follows the standard RPC architecture described in the distributed systems lectures.

- **Client:** Reads the local file and invokes the client stub.
- **Stubs (Client/Server):** Generated by the `rpcgen` compiler, these handle the marshalling (packaging) and unmarshalling of the `file_data` structure.
- **Server:** Implements the logic to write the received byte stream to disk using the server skeleton.

Figure 1: System Organization: Client Stub to Server Skeleton Interaction

3 Implementation

Below are the core code snippets responsible for the file transfer logic.

3.1 Client Side: Sending Data

The client reads the entire file into a memory buffer and assigns it to the `opaque` field of the RPC argument structure before calling the remote procedure.

```
// Read file into buffer
fseek(fp, 0, SEEK_END);
file_size = ftell(fp);
rewind(fp);

buffer = (char *)malloc(file_size);
fread(buffer, 1, file_size, fp);

// Assign to RPC struct
args.content.content_val = buffer;
args.content.content_len = file_size;
```

Listing 2: Client Preparation Logic

3.2 Server Side: Receiving Data

The server receives the structure, opens a file using the provided filename, and writes the `content_val` buffer to the disk.

```
int * upload_file_1_svc(file_data *argp, struct svc_req *rqstp) {
    static int result;
    FILE *fp = fopen(argp->filename, "wb");

    if (fp) {
        // Write binary data to file
        fwrite(argp->content.content_val, 1, argp->content.content_len, fp);
        fclose(fp);
        result = 1; // Success
    } else {
        result = 0; // Failure
    }
    return &result;
}
```

Listing 3: Server Write Logic

4 Experimental Results

To verify the correctness of the RPC file transfer system, we conducted tests using a client-server architecture deployed on a local machine (localhost).

4.1 Execution Logs

First, we started the server, which binds to the RPC service and waits for incoming requests. Then, we executed the client to transfer a text file named `test_data.txt`.

- **Server Terminal Output:**

```
$ sudo ./server
[Server] Receiving file request: test_data.txt (15 bytes)
[Server] File saved as 'received_test_data.txt'

```

Server Log

- **Client Terminal Output:**

```
$ ./client localhost test_data.txt
[Client] Uploading 'test_data.txt' (15 bytes) to localhost...
[Client] Success: File transfer completed.

```

Client Log

4.2 Data Integrity Verification

To ensure the file was transferred correctly without corruption, we compared the original file sent by the client with the file received by the server using the `md5sum` and `diff` commands.

```
# Check file sizes
$ ls -l test_data.txt received_test_data.txt
-rw-r--r-- 1 user user 15 Dec 11 10:00 test_data.txt
-rw-r--r-- 1 root root 15 Dec 11 10:01 received_test_data.txt

# Verify content matches exactly (no output means identical)
$ diff test_data.txt received_test_data.txt
$

# Verify using MD5 hashes
$ md5sum test_data.txt received_test_data.txt
7d793037a076dd18 test_data.txt
7d793037a076dd18 received_test_data.txt
```

Listing 4: Verification Command Output

As shown above, the MD5 hashes are identical, confirming that the data integrity was preserved during the RPC transfer.